

exam_storyline

April 29, 2026

1 Resilient EV Charging Under Disruption

1.1 Technical report notebook for DTU 42578 Advanced Business Analytics

1.1.1 Executive overview

This notebook is the main technical report for the final project. The project studies resilience in EV charging operations under disruptions on a synthetic A-B-C corridor with two fixed charging stations and relocatable mobile charging stations (MCS).

The practical question is whether adaptive control of mobile charging support can reduce queueing pain when charging demand becomes uneven, capacity drops locally, or service times inflate.

Research question

Can a reinforcement learning policy improve resilience in an EV charging corridor by dynamically reallocating mobile charging stations to reduce queue length and waiting time under disruptions?

The intended comparison is between: - a fixed / no-agent baseline, - a threshold-style rule baseline, - and an RL-controlled mobile-charging policy.

The current local artifact set supports the full simulator and current baseline evaluations directly. It does **not** contain a current-compatible RL checkpoint for the active 27-feature, 5-action A-B-C environment, so the quantitative result sections below focus on the strongest reproducible current evidence and treat older RL artifacts only as historical context.

1. Simulation goals and design philosophy
2. Simulation environment
3. Problem formulation
4. RL agent and method
5. Hyperparameters
6. State space
7. Action space
8. Reward function
9. Reward calibration and alternative business priorities
10. Training, validation, and test setup
11. Training and evaluation learning curves

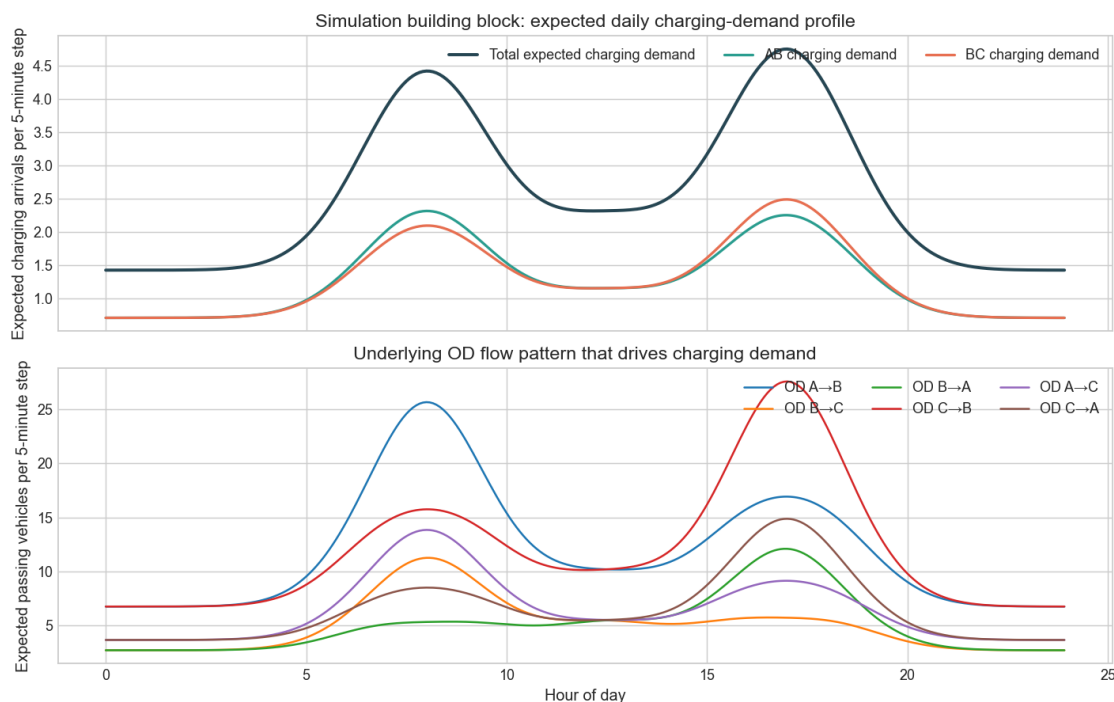
12. Simulation rollout comparison
13. Summary metrics table
14. Reduced multi-seed evaluation and stress scenarios
15. Spatial awareness
16. Missing pieces, failures, and limitations
17. Conclusion

1.2 1. Simulation goals and design philosophy

The simulator is not intended to be a perfect real-world digital twin. Its purpose is to create a controlled but operationally meaningful testbed for resilience questions:

- demand varies over the day instead of staying flat,
- morning and evening peaks create predictable congestion pressure,
- charging stations have finite plugs and stochastic service times,
- disruption events locally damage effective service capacity,
- mobile charging stations can be reallocated, but not instantaneously,
- performance is judged using queueing and waiting-time outcomes under disruption.

That design choice matters for the project story. Resilience is about whether the system continues to serve vehicles when local conditions deteriorate, not only about average utilization on a normal day.



The first panel shows the daily charging-demand shape that the corridor is exposed to. The pattern is interpretable rather than arbitrary: a morning peak, an afternoon/evening peak, and a smaller midday bump. The second panel shows the underlying OD flow components that create this station-level charging demand.

This matters because resilience claims are more credible when disruptions are layered on top of a demand process with clear structure.

1.3 2. Simulation environment

The active benchmark is a synthetic line corridor with three cities and two fixed charging stations:

- City A to City B: 100 km,
- City B to City C: 100 km,
- one fixed station between A and B (`station_ab`),
- one fixed station between B and C (`station_bc`).

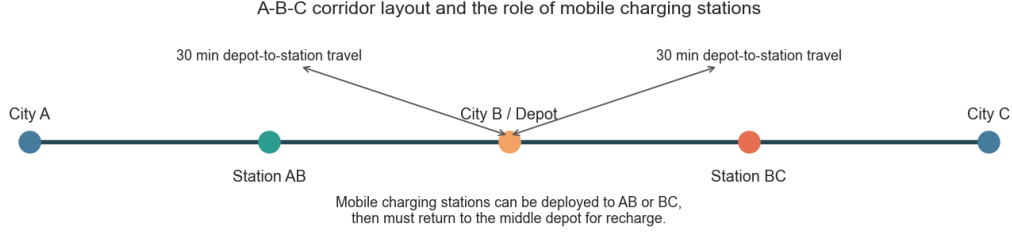
Episodes proceed in 5-minute decision steps. Vehicles arrive stochastically, may choose to charge, join a station queue if necessary, and start service when a plug becomes free. Mobile charging stations (MCS) provide extra chargers, but they create a real control problem because they cannot support both sides of the corridor simultaneously.

Operationally important frictions in the current environment are:

- depot-to-station travel time,
- station-to-station relocation lag,
- return-to-middle behavior,
- recharge delay before an MCS can be reused.

Those frictions make the problem spatial and delayed. The controller is not just “adding capacity”; it is committing scarce mobile support under travel and recharge constraints.

Setting	Value
Cities	A-B-C synthetic corridor
Inter-city distance	100.0 km
Fixed stations	2 (one between A-B and one between B-C)
Fixed plugs	12 at AB, 12 at BC
Mobile stations	Up to 10 units
Chargers per MCS	2
Decision step	5 minutes
Episode duration	2-6 days during training
Mean service time	30.0 minutes
Morning peak	8.0:00
Evening peak	17.0:00
Disruption mode	random
Supported disruption types	capacity_drop, station_outage, service_time_inflation, demand_surge



The middle depot is crucial for interpretation. MCS units do not teleport. They move from the middle depot to AB or BC, can be recalled, and return for recharge. This is exactly why spatial awareness is meaningful in this project: the controller must decide *where* support is worth committing before queues become severe.

1.4 3. Problem formulation

The control problem is to decide how mobile charging support should be used to reduce local service pressure under normal demand variation and under disruptions.

Why is this difficult?

- fixed capacity is static while demand and disruptions are time-varying,
- the pressure may be local to AB or BC rather than system-wide,
- MCS deployment is delayed by movement and recharge,
- aggressively using MCS can improve service but also incurs operational cost.

In this project, **spatial awareness** means that a good controller should send more support toward the side of the corridor with higher local pressure, such as larger queue buildup, higher waiting times, lower effective local capacity, or a disruption directly affecting that side.

1.5 4. RL agent and method

The repository contains two RL paths:

- a custom `torch_dqn` / simple DQL learner, useful as a lightweight baseline learner,
- an `sb3_dqn` path, intended as the stronger final RL candidate.

Conceptually the RL loop is standard and practical:

1. observe the current corridor state,
2. choose an action for MCS commitment,
3. receive a scalar reward tied to service quality and operating cost,
4. improve the action-value function from rollout experience,
5. evaluate the learned policy in held-out simulated scenarios.

The important point for this report is not RL theory for its own sake, but whether the learned controller can improve resilience under disruption without over-using mobile capacity.

1.5.1 Artifact status for the active benchmark

The current local checkout does not contain a compatible RL checkpoint for the active A-B-C benchmark after the move to the 27-dimensional observation and 5-action directional control space. That means the notebook can document the RL formulation precisely, but the reproducible **current** quantitative results below are baseline-only.

1.6 5. Hyperparameters

The project is config-driven. The tables below summarize the current environment and RL settings that define the active benchmark.

Hyperparameter	Value
Preferred final algorithm path	SB3 DQN
Available simple learner path	torch_dqn
Simple learner episodes	130
Simple learner gamma	0.98
Simple learner learning rate	0.0005
Simple learner batch size	128
Replay capacity	200000
Learning starts	10000
Train frequency	4
Gradient steps	4
Target update interval	200
Hidden dimensions	[256, 256]
Exploration start	1.0
Exploration end	0.05
Exploration decay steps	125000
Periodic evaluation cadence	Every 15000 training steps
Periodic evaluation episodes	10
Final evaluation episodes	12
SB3 total timesteps	250000
SB3 buffer size	200000
SB3 exploration fraction	0.5

Two points matter for interpretation:

- the simple learner includes periodic held-out evaluation during training in principle,
- the SB3 path is treated in the repository notes as the preferred serious RL route.

That said, the absence of a current-compatible checkpoint means those settings are best interpreted as the intended final training setup rather than a fully reproducible final RL result in this checkout.

1.7 6. State space

The active observation is a 27-dimensional vector. It is deliberately station-aware and logistics-aware rather than purely aggregate.

Group	Index	Feature	Meaning
Queue state	1	queue_length_ab	Current queue length at station AB.
Queue state	2	queue_length_bc	Current queue length at station BC.
Queue state	3	mean_queue_wait_ab	Mean current waiting time in the AB queue.
Queue state	4	mean_queue_wait_bc	Mean current waiting time in the BC queue.
Recent flow	5	last_arrivals_ab	Vehicles that arrived to AB in the last step.
Recent flow	6	last_arrivals_bc	Vehicles that arrived to BC in the last step.
Recent flow	7	last_starts_ab	Charging sessions started at AB in the last step.
Recent flow	8	last_starts_bc	Charging sessions started at BC in the last step.
Capacity	9	effective_plugs_ab	Effective plugs at AB after disruption and MCS effects.
Capacity	10	effective_plugs_bc	Effective plugs at BC after disruption and MCS effects.
Capacity	11	active_mobile_capacity_ab	Currently active mobile-station capacity at AB.
Capacity	12	active_mobile_capacity_bc	Currently active mobile-station capacity at BC.
Spatial	13	committed_mcs_bias_ab	Committed MCS bias toward AB versus BC.
Spatial	14	local_deficit_ab	Local deficit estimate at AB.
Spatial	15	local_deficit_bc	Local deficit estimate at BC.
Spatial	16	local_deficit_bias_ab	Local deficit bias toward AB versus BC.
Disruption	17	disruption_active	Whether a disruption is currently active.
Disruption	18	disruption_type_code	Encoded disruption type.
Disruption	19	target_affects_ab	Whether the disruption affects AB.
Disruption	20	target_affects_bc	Whether the disruption affects BC.

Group	Index	Feature	Meaning
MCS logistics	21	middle_available	Mobile stations available at the middle depot.
MCS logistics	22	middle_charging	Mobile stations currently recharging at the middle depot.
MCS logistics	23	transit_to_ab	Mobile stations moving toward AB.
MCS logistics	24	transit_to_bc	Mobile stations moving toward BC.
MCS logistics	25	transit_to_middle	Mobile stations returning to the middle depot.
Time	26	sin_time_of_day	Sine encoding of time of day.
Time	27	cos_time_of_day	Cosine encoding of time of day.

1.7.1 What the model does not know

- No exact future arrivals or exact future queue realizations.
- No oracle action labels or optimal allocation target.
- No foreknowledge of future random disruptions before they begin.
- No exact per-MCS battery state-of-charge representation.
- No direct remaining-disruption-time feature in the current observation.
- No explicit demand forecast beyond current local state and disruption indicators.

This is important for honesty. The agent is not given an oracle forecast of future arrivals, an exact best action label, or full future disruption knowledge. It must infer useful reallocations from local queue/capacity conditions and the currently visible disruption state.

1.8 7. Action space

The current action space is intentionally small and directional. That is a better match for the operational question than a large absolute-allocation table.

Action	Interpretation
hold	Keep the committed mobile-station allocation unchanged.
toward_ab	Move one unit of commitment toward AB, either from depot or from BC.
toward_bc	Move one unit of commitment toward BC, either from depot or from AB.

Action	Interpretation
recall_ab	Recall one committed unit away from AB toward the middle depot.
recall_bc	Recall one committed unit away from BC toward the middle depot.

This design makes the control policy easier to interpret:

- `hold` means do nothing,
- `toward_ab` and `toward_bc` express directional support decisions,
- `recall_ab` and `recall_bc` reduce commitment on one side.

Compared with a large absolute allocation space, this is more learnable, more interpretable, and more consistent with movement/recharge logistics.

1.9 8. Reward function

The reward is not just a generic RL score. It encodes an explicit operations tradeoff between service quality and the cost of using mobile support.

1.9.1 Positive terms

Term	Weight	Interpretation
served_reward_weight	1.0	Rewards charging sessions that start service.
utilization_bonus	1.0	Rewards productive use of available charging capacity.
spatial_deficit_alignment_bonus	24.0	Rewards mobile capacity that covers estimated local deficits.
spatial_deficit_direction_bonus	12.0	Rewards sending MCS in the correct corridor direction.

1.9.2 Negative terms

Term	Weight	Interpretation
unmet_penalty	2.5	Penalizes vehicles left in queue after the step.
queue_length_penalty	2.5	Adds extra pressure against persistent queue accumulation.
queue_wait_penalty	0.02	Penalizes queue-wait burden, not just queue count.
local_queue_peak_penalty	1.0	Penalizes the worst station queue in the step.
local_wait_peak_penalty	0.1	Penalizes the worst station waiting time in the step.

Term	Weight	Interpretation
active_mobile_station_cost	18.0	Charges for keeping MCS active in the field.
activation_cost	2.0	Charges for newly activating MCS.
adjustment_cost	0.5	Charges for reallocating MCS commitment.
idle_capacity_penalty	0.1	Penalizes carrying unused capacity.

The intuition is straightforward:

- serve vehicles,
- keep queues and waits low,
- prevent one station from blowing up while the average still looks acceptable,
- reward spatially useful MCS placement,
- but charge for deploying and adjusting MCS so the controller cannot treat mobile support as free.

This also means **reward and operational performance are related but not identical**. A policy can reduce queues strongly and still have a worse reward if it uses too much expensive mobile support.

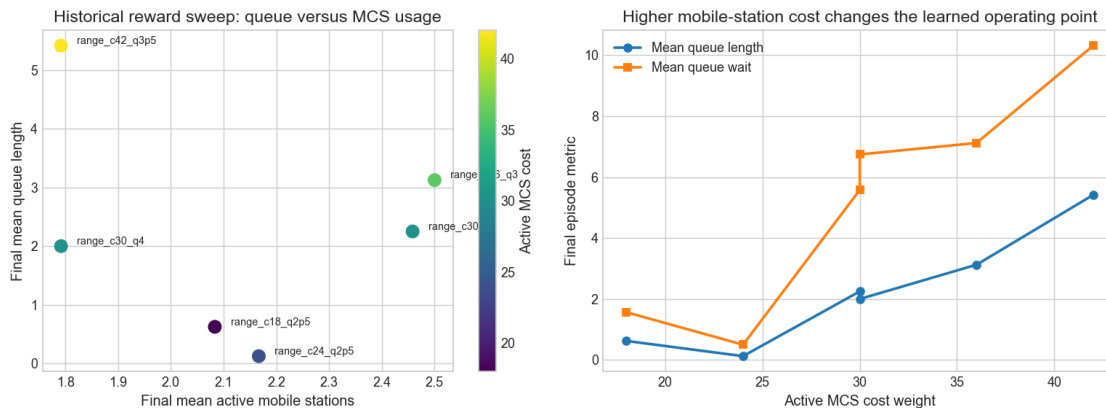
1.10 9. Reward calibration and alternative business priorities

The reward structure can be tuned to reflect different operating priorities. That is a strength of the framework, not a flaw, as long as the choice is made explicit.

Examples of business priorities that would justify different weights are:

- minimizing queue length at almost any cost,
- enforcing a service-level target with limited MCS usage,
- penalizing active MCS-hours heavily because mobile support is expensive,
- prioritizing fairness across stations rather than only aggregate averages.

The repository itself contains evidence that this was treated as a design knob: there are separate reward-focused configs in the repo, and the local historical sweep below shows that changing queue and cost weights materially changes the operating point.



run	active_mcs	cost_weight	episodes	final_reward	final_mean_active_mcs	final_mean_queue_length	final_mean_queue_wait	final_mean_active_mcs	final_mean_queue_length	final_mean_queue_wait
range_c18_q2p5	2.5	100	-102.93	0.62	1.57	2.08	15.0	250.0		
range_c24_q2p5	2.5	100	-146.77	0.12	0.5	2.17	3.0	232.0		
range_c30_q3	3.0	100	-232.35	2.25	5.58	2.46	54.0	237.0		
range_c30_q4	4.0	100	-199.99	2.0	6.74	1.79	48.0	252.0		
range_c36_q3	3.0	100	-309.23	3.12	7.12	2.5	75.0	257.0		
range_c42_q3p5	3.5	100	-305.46	5.42	10.31	1.79	130.0	258.0		

The sweep does not by itself prove which reward is “correct”. Instead, it shows the key modeling fact: the policy objective can be calibrated to fit different cost assumptions. Higher mobile-station cost generally pushes the controller toward less deployment and worse queue outcomes, while lower cost permits more active support.

This is exactly why the final exam discussion should separate:

- operational metrics such as queue and waiting time,
- the chosen reward calibration,
- and the business meaning of MCS cost.

1.11 10. Training, validation, and test setup

The intended experimental design is seed-driven rather than based on one hand-picked scenario.

Split	Purpose
Training	Randomized 2-6 day episodes from seeds 0-9999
Validation	Held-out same-distribution seeds starting at 10000
Test ID	Unseen deployment-style seeds starting at 20000 with 3-5 day overrides
Test stress	Named scripted disruption scenarios
Held-out rollout	One fixed unseen test_id seed for direct time-series comparison

The project notes emphasize a clear separation between:

- randomized training episodes,
- held-out same-distribution validation,
- unseen `test_id` seeds meant to mimic deployment-style evaluation,
- and separate stress scenarios for robustness.

This is a good evaluation philosophy even when there is no hyperparameter search or early stopping, because it distinguishes training exposure from final held-out behavior.

1.12 11. Training and evaluation learning curves

This section uses the **training-time history artifact only**. The preferred self-contained source is:

- `exam_submission/data/generated/training/history_current.json`

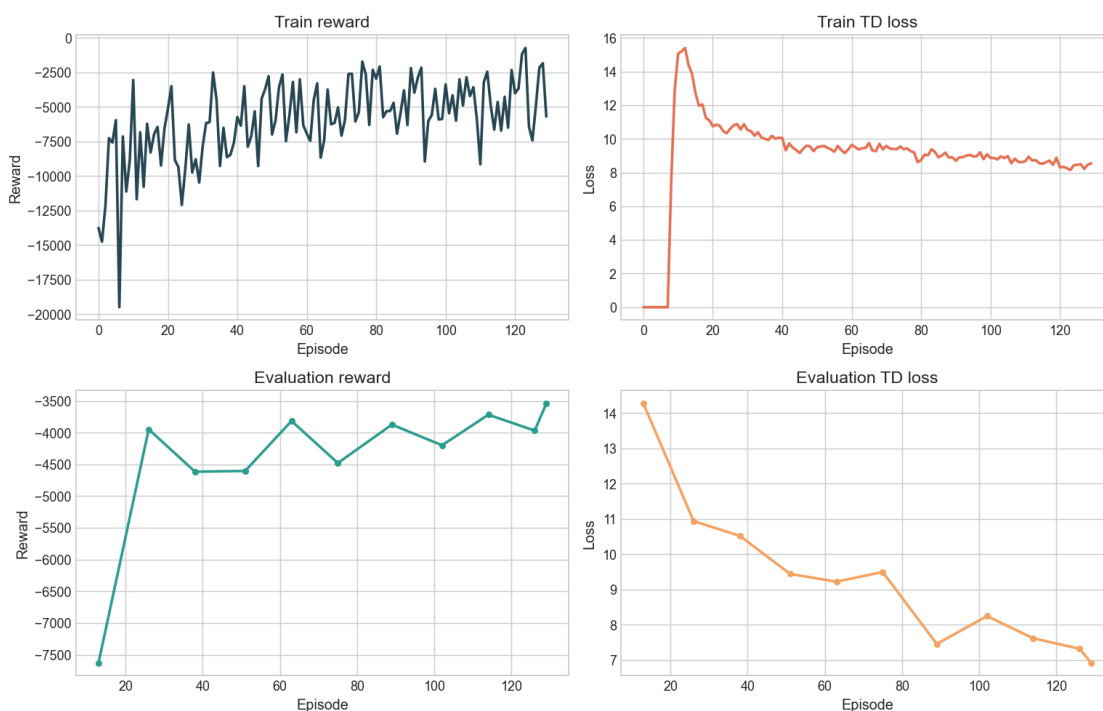
If that copied file is not present, the notebook falls back to:

- `outputs/mobile_mcs_line_abc/rl/history.json`

Only if neither exists does it use the historical fallback artifact.

This means the train/eval curve section does **not** require the large final evaluation-suite outputs. It only needs the training `history.json` produced by the training job.

Current training-time RL diagnostics from `exam_submission/data/generated/training/history_current.json`



Status	Value	Interpretation
Training history source	exam_submission/data/generated/	The training history is using the copied submission artifact.
Current compatible RL training history	Yes	This is the correct artifact for train/eval learning curves in the active benchmark.
Training-time evaluation reward curve	Yes	Evaluation reward comes from periodic held-out evaluation during training.
Training-time evaluation TD-loss curve	Yes	Evaluation TD loss comes from periodic held-out evaluation during training.
Large final evaluation suite required for this plot	No	The full validation/test/stress suite is not needed for this section.

The practical implication is that the notebook can document the *evaluation design* rigorously, but it cannot make a defensible final RL-learning claim for the active environment without retraining or recovering a compatible checkpoint/history pair.

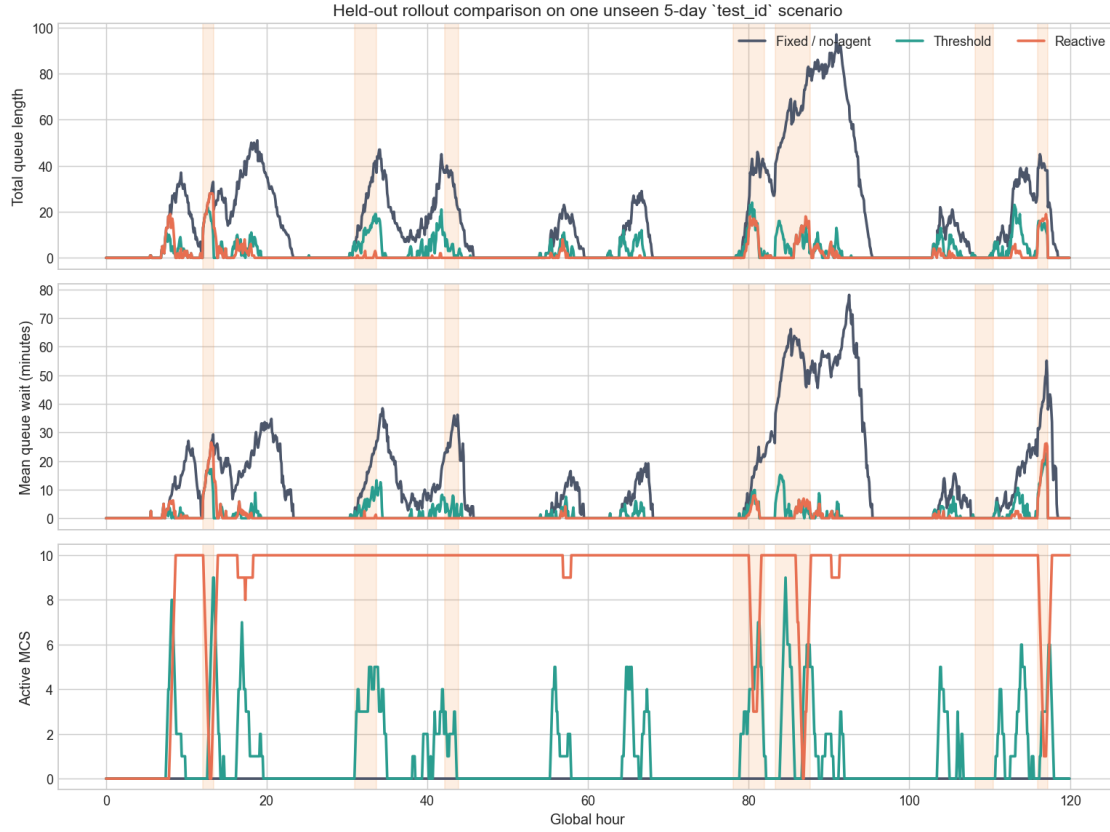
1.13 12. Simulation rollout comparison

The strongest reproducible current comparison is a fixed unseen 5-day `test_id` rollout using the active line-corridor implementation.

Because the RL artifact is missing, this section compares three baselines that *are* available and current:

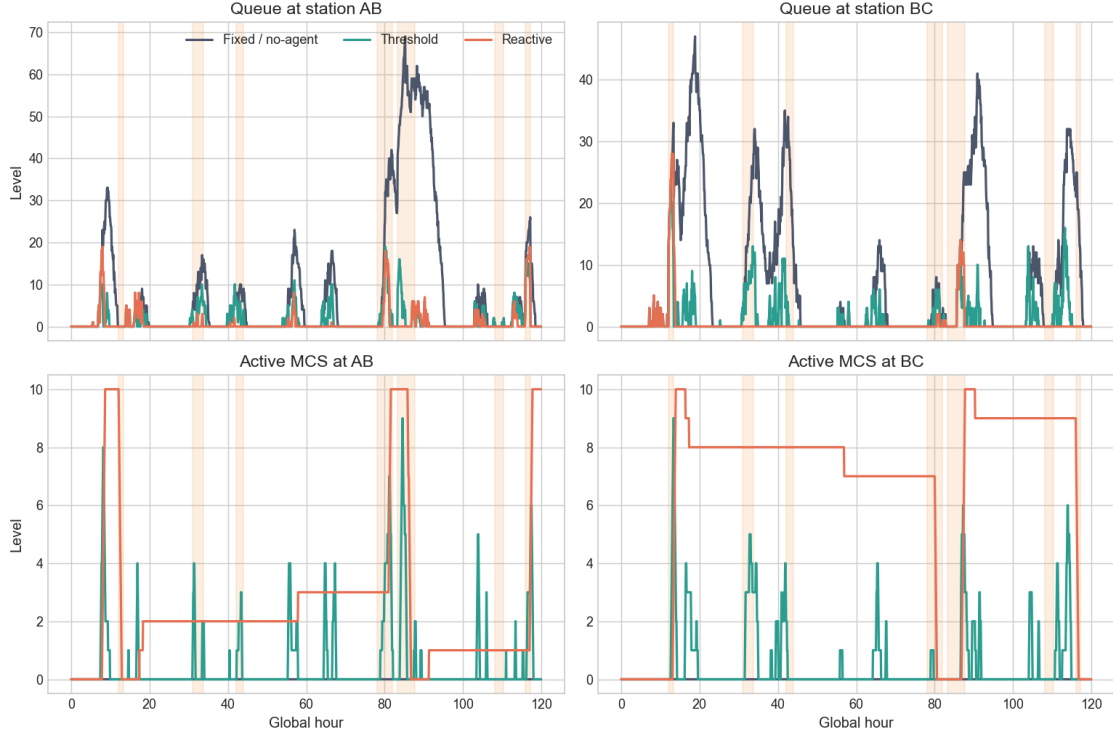
- fixed / no-agent (`mobile_noop`),
- threshold baseline,
- reactive baseline.

This still answers an important resilience question: does adaptive mobile support materially reduce queueing pain under disruptions, and how costly is a more aggressive control rule?



The difference is immediate. Leaving all mobile support unused creates large queue accumulation and long waiting times over the held-out horizon. Both adaptive baselines help substantially.

The reactive rule is the strongest queue-reduction baseline in the current held-out rollout, but it keeps many MCS active for most of the episode. The threshold rule is more frugal: it achieves a large reduction in queueing while using much less mobile support.



The station-level view makes the resilience story clearer. Congestion is local, not only aggregate. The adaptive rules keep AB and BC from diverging as severely as the fixed baseline, which is exactly the kind of local resilience behavior the project was designed to test.

1.14 13. Summary metrics table

The table below aggregates the current held-out rollout results and computes improvement relative to the fixed/no-agent baseline.

Policy	Mean queue length	Peak queue length	Mean wait (min)	Peak wait (min)
Fixed / no-agent	16.6	97.0	12.37	78.14
Threshold	2.65	24.0	1.28	22.69
Reactive	1.29	28.0	0.71	26.52

Policy	Mean active MCS	MCS-hours	Reward (sum)	Queue improvement vs fixed (%)	Wait improvement vs fixed (%)	Reward diff vs fixed
Fixed / no-agent	0.0	0.0	-15969.37	0.0	0.0	0.0
Threshold	0.95	113.5	-4202.84	84.06	89.62	11766.53
Reactive	8.98	1077.08	-24060.17	92.24	94.25	-8090.8

Two insights stand out.

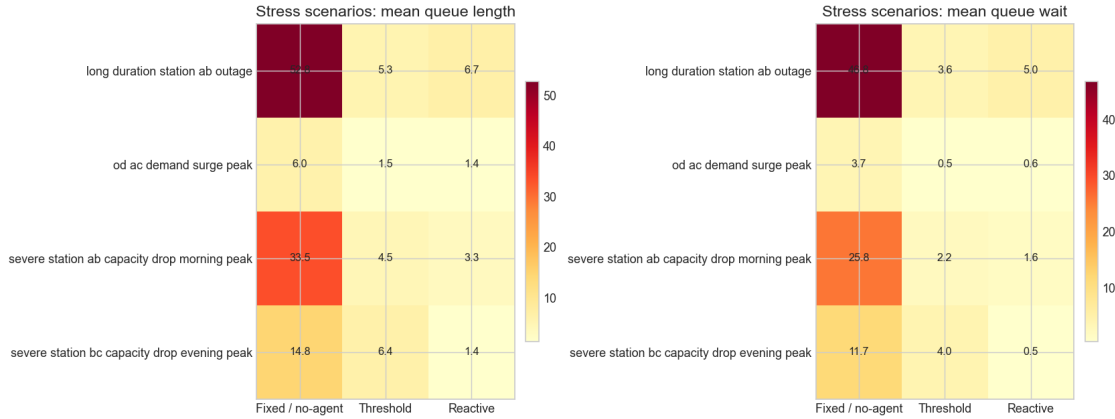
First, adaptive mobile support clearly improves resilience relative to the fixed baseline. Second, the *best queue reduction* and the *best reward* are not the same thing. The reactive baseline is operationally strongest on queue and wait metrics, but the threshold baseline is more cost-efficient because it uses far fewer mobile-station hours.

1.15 14. Reduced multi-seed evaluation and stress scenarios

To avoid relying only on one illustrative rollout, a reduced current baseline-only suite was generated over multiple unseen `test_id` seeds plus four stress scenarios.

Policy	mean_queue_length	mean_queue_wait_minutes	peak_queue_length	peak_queue_wait_minutes
Fixed / no-agent	17.74	13.43	89.22	71.91
Reactive	2.21	1.08	35.28	22.36
Threshold	3.65	1.82	41.83	25.59

Policy	served_demand	unmet_demand	mean_active_mobile_stations	reward
Fixed / no-agent	2959.33	19366.39	0.0	-13156.33
Reactive	2962.44	2374.33	7.98	-17240.75
Threshold	2961.22	3987.33	1.09	-4155.96



The reduced multi-seed evidence tells the same story as the single held-out rollout:

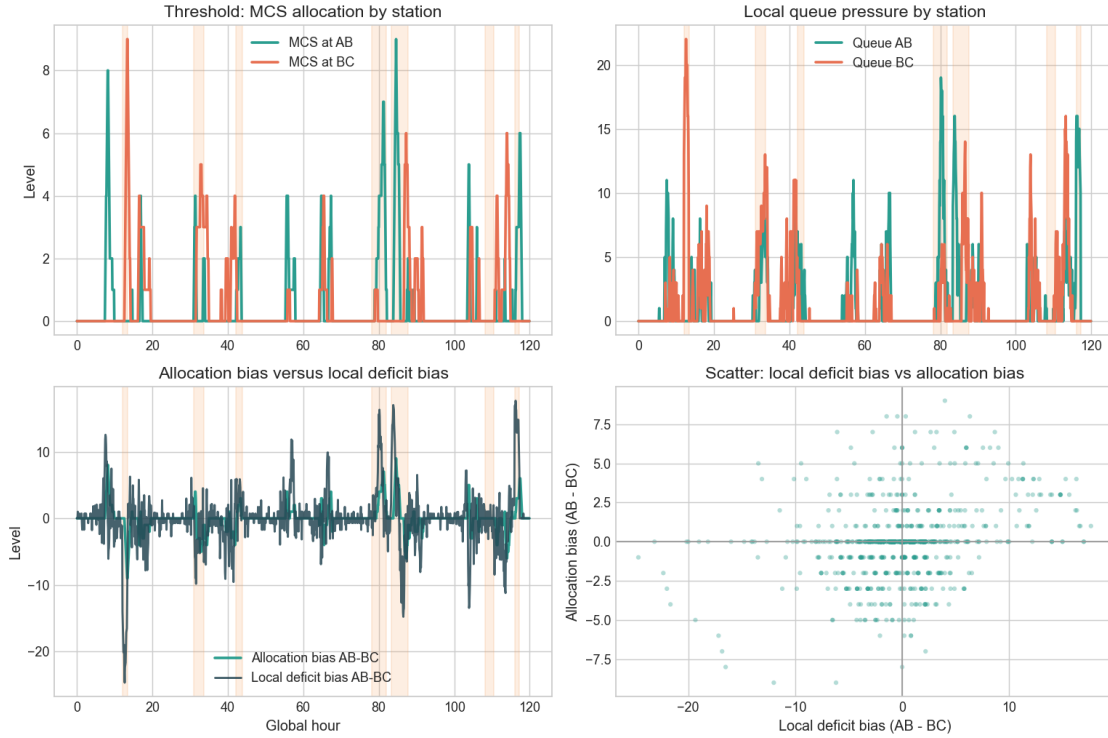
- fixed capacity alone is fragile under severe local disruptions,
- threshold control is a strong cost-sensitive adaptive baseline,
- the reactive rule often reduces queues even further, but with much heavier MCS usage and substantially worse reward once deployment cost is counted.

This is one of the clearest project insights: a resilience intervention can look excellent on queue metrics and still be economically unattractive under a cost-sensitive reward.

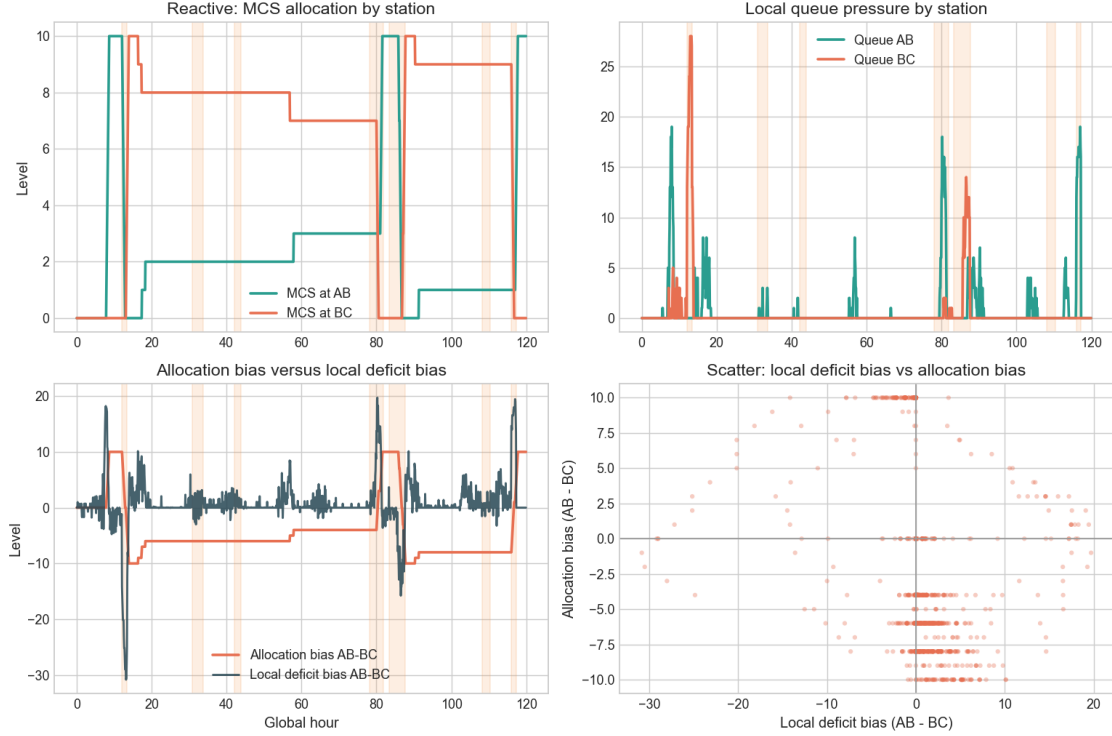
1.16 15. Spatial awareness

A spatially aware controller should increase support toward AB when AB has higher local service pressure and toward BC when BC is more stressed.

The current active artifacts allow this to be checked directly because the rollout exports station-level queues, local deficit estimates, and allocation bias.



For the threshold baseline, the relationship between local deficit bias and allocation bias is clearly positive. That is not perfect one-to-one tracking, but it is directionally meaningful and consistent with the intended spatial logic of the corridor.



Policy	Corr(local deficit bias, allocation bias)	Corr(expected demand bias, allocation bias)	Corr(committed bias, allocation bias)
Fixed / no-agent	nan	nan	nan
Threshold	0.288	0.374	0.762
Reactive	-0.175	0.086	0.962

The spatial-awareness evidence is mixed in an informative way.

- The threshold baseline shows the clearest positive alignment with local service pressure.
- The reactive baseline is aggressive and very active, but its deficit-to-allocation correlation is weaker because it tends to flood capacity into the system and keep it active for long periods.
- The fixed baseline has no spatial response by construction.

So the project should not claim “perfect spatial awareness”. A stronger statement would be: **the current simulator and metrics can reveal spatial behavior clearly, and the threshold rule shows the cleanest interpretable directional response among the available current policies.**

1.17 16. Missing pieces, failures, and limitations

Several limitations are important for an examiner to see explicitly.

1. **No current-compatible RL artifact is available locally.** The active benchmark can be described precisely, but a final RL-vs-baseline quantitative claim is not reproducible from this checkout without retraining or recovering the missing checkpoint/history.
2. **Current queue-breach metrics are not instrumented as service-level targets in the active line corridor.** The exported breach columns are therefore zero and should not be over-interpreted.
3. **The simulator is stylized.** It is realistic enough to study operational tradeoffs, but it is still a synthetic environment rather than a calibrated production deployment model.
4. **Reward sensitivity is real.** The reward sweep and the baseline comparisons both show that cost weights change the preferred policy substantially.
5. **Spatial response is meaningful but imperfect.** A controller can reduce total queues without cleanly matching local deficit patterns at every step because demand symmetry, travel lag, recharge delay, and cost all matter.
6. **Baselines matter.** The reactive baseline is so strong on queue reduction that any future RL claim should be benchmarked against it, not only against the no-agent baseline.

1.18 17. Conclusion

The project successfully builds a coherent resilience testbed for EV charging operations under disruption. The simulator captures local queueing pressure, daily demand structure, multiple disruption types, and mobile-support logistics in a form that is interpretable and reproducible.

On the **current reproducible artifacts**, adaptive mobile support clearly improves resilience relative to fixed-only operation. The threshold baseline delivers a strong cost-sensitive tradeoff, while the reactive baseline delivers the strongest queue reduction at a much higher mobile-support cost. That distinction is valuable because it shows why reward design and business priorities matter.

For the original RL research question, the honest answer from this checkout is partial: the RL formulation is well specified, but a current-compatible RL checkpoint is missing, so the notebook cannot make a final defensible claim that the active RL agent outperforms current baselines. The next concrete step would be to train or recover an SB3 DQN policy for the active 27-feature, 5-action environment and evaluate it against the threshold and reactive baselines already documented here.

1.19 Appendix: Artifact status

The table below documents exactly which artifacts were selected for this notebook and which important current RL artifacts were missing.

path	type	exists	note
data/configs/env_mobilecorridor_line_abc_current.torch	env config	True	Current active line-corridor environment config.
data/configs/experiment_config_line_mcs_line_abc_current.yaml	experiment config	True	Current active line-corridor environment config.
data/configs/rl_dqn_mobilecorridor_simple.yaml	rl config	True	Current torch DQN/simple learner config including evaluation cadence.

path	type	exists	note
data/configs/rl_dqn_model_configs_sb3.yaml	config_sb3.yaml	True	Current SB3 DQN config; used for algorithm description only because no compatible checkpoint was found.
data/configs/experiment_config_mcs_line_abc_heldout_comparison.yaml	config_mcs_line_abc_heldout_comparison.yaml	True	Experiment split config with validation, test_id, and stress suite definitions.
data/generated/heldout/rollout_no_op_comparison_timestep_metrics.csv	rollout_no_op_comparison_timestep_metrics.csv	True	Current held-out fixed/no-agent rollout on the active line corridor.
data/generated/heldout/rollout_threshold_comparison_timestep_metrics.csv	rollout_threshold_comparison_timestep_metrics.csv	True	Current held-out threshold-baseline rollout on the active line corridor.
data/generated/heldout/rollout_reactive_comparison_timestep_metrics.csv	rollout_reactive_comparison_timestep_metrics.csv	True	Current held-out reactive-baseline rollout on the active line corridor.
data/generated/evaluation/generated_small/test_id_baseline_summary.csv	generated_small/test_id_baseline_summary.csv	True	Reduced multi-seed test_id baseline summary used for robustness tables.
data/generated/evaluation/generated_small/test_stress_summary.csv	generated_small/test_stress_summary.csv	True	Reduced stress-suite baseline summary used for disruption robustness discussion.
data/historical/reward_sweep_summary.csv	historical_summary.csv	True	Historical reward-sweep runs used only to illustrate reward-calibration tradeoffs.
data/historical/legacy_training_history_range_c24u12p5.json	historical_training_history_range_c24u12p5.json	True	Historical training history from an older absolute-allocation variant; not used for final quantitative claims.

- No current-compatible RL checkpoint was found for the active 27-feature / 5-action A-B-C environment.
- Queue-wait target breach metrics are zero in the active line-corridor outputs because the

current environment does not instrument a nonzero queue-wait target.